Proyecto 7: Copias de Seguridad con Bacula

Abel García Domínguez
IES Rafael Alberti — Curso Especialización Ciberseguridad

Fundamentación Teórica

Uno de los aspectos más importantes de las copias de seguridad empresariales es la **automatización**. Si realizamos las copias de seguridad de forma manual, supone una pérdida de tiempo importante, además de la alta posibilidad de olvidar alguna. En lugar de configurar cada equipo individualmente, Bacula gestiona centralizadamente con interfaz **Baculum**.

Objetivos del Proyecto

Diseñar una estrategia de copias de seguridad y ponerla en producción con **Bacula + Baculum** para clientes Linux y Windows.

Estrategia de Copias de Seguridad

Estrategia semanal híbrida (completa + incremental + diferencial)

> Optimiza la relación entre tiempo de backup y uso de almacenamiento.

Día	Tipo Backup	Descripción
Lunes	Completa	Todos los datos del cliente
Martes	Incremental	Cambios desde Lunes
Miércoles	Diferencial	Cambios desde completa (Lunes)
Jueves	Incremental	Cambios desde Martes
Viernes	Incremental	Cambios desde Jueves
Sábado	Incremental	Cambios desde Viernes
Domingo	Incremental	Cambios desde Sábado

Justificación

- **Full semanal** → minimiza el tiempo de restauración.
- **Diferenciales** → rápidas y menos costosas que una completa.
- **Incrementales diarias** → ahorran espacio al registrar solo los cambios más recientes.
- **Retención:** 4 semanas + copias mensuales.

Instalación del Entorno

Acceso remoto al servidor

La instalación se realiza desde el equipo local mediante una conexión **SSH** al servidor Ubuntu Server, lo que permite administrar el sistema de forma remota sin necesidad de acceso físico a la máquina.

```
ssh ubuntu@<IP_del_servidor>
```

Fase 1: Stack LAMP + Bacula

Instalar stack LAMP

```
sudo apt install apache2 mysql-server mysql-client php php-mysql -y
```

> *Explicación:* Apache y PHP son necesarios para servir la interfaz web de Baculum. MySQL actúa como catálogo donde Bacula almacena el historial de jobs y volúmenes.

Instalar paquetes Bacula Community completos:

```
sudo apt install bacula bacula-client bacula-common-mysql bacula-director-mysql
bacula-server -y
```

> *Explicación:* Se instalan todos los componentes del stack Bacula:
>
> - **bacula-director**: componente central que gestiona y coordina los jobs de backup.
> - **bacula-storage**: gestiona los volúmenes donde se almacenan las copias.
> - **bacula-client**: se ejecuta en cada cliente para enviar los datos al director.
> - **bacula-common-mysql**: librería de integración con MySQL para el catálogo.

Fase 2: Configuración de Bacula

Configuración del servidor Ubuntu

En el servidor se configuró el fichero `/etc/bacula/bacula-dir.conf` para definir el **Director**, los **Jobs**, los **FileSets**, los **clientes**, el **Storage Daemon** y la planificación de copias de seguridad.

Se definieron dos clientes:

- Un cliente Linux llamado `debian1`
- Un cliente Windows llamado `windows-client-fd`

Para el cliente Linux se configuró el siguiente bloque:

```
  GNU nano 7.2                          bacula-dir.conf
# CLIENTES
# ----------------------------------------------------------

# Cliente Linux
Client {
  Name = debian1
  Address = 192.168.1.143        # <-- IP del equipo Linux
  FDPort = 9102
  Catalog = MyCatalog
  Password = "mmHKEHDLEbk-FZQHvCqJ6sAmARXPPMyur"   # <-- d
  File Retention = 60 days
  Job Retention = 6 months
  AutoPrune = yes
  TLS Enable = no
  TLS Require = no
}
```

Para el cliente Windows se añadió un bloque equivalente en el Director, indicando su dirección IP, puerto de escucha y contraseña compartida con el equipo Windows:

```
# Cliente Windows
Client {
  Name = windows-client-fd
  Address = 192.168.1.144     # <-
  FDPort = 9102
  Catalog = MyCatalog
  Password = "SevillaFC"  # <-- de
  File Retention = 60 days
  Job Retention = 6 months
  AutoPrune = yes
}
```

Se creó además el job `BackupClientLinux`, asociado al cliente Debian y al FileSet `Linux Set`. También se creó un job específico para el cliente Windows:

```
# Backup cliente Linux
Job {
  Name = "BackupClientLinux"
  JobDefs = "DefaultJob"
  Client = debian1
  FileSet = "Linux Set"
}

# Backup cliente Windows
Job {
  Name = "BackupClientWindows"
  JobDefs = "DefaultJob"
  Client = windows-client-fd
  FileSet = "Windows Set"
}
```

El FileSet del cliente Linux fue configurado para realizar copia de seguridad de los directorios más importantes del sistema:

```
# FileSet para el cliente Linux
FileSet {
  Name = "Linux Set"
  Include {
    Options {
      signature = MD5
      compression = GZIP
    }
    File = /home
    File = /etc
    File = /var
  }
  Exclude {
    File = /var/lib/bacula
    File = /proc
    File = /tmp
    File = /sys
    File = /.journal
    File = /.fsck
  }
}
```

De esta forma, fueron incluidos los directorios de usuarios, configuración del sistema y datos de servicios, excluyendo rutas temporales o virtuales que no deben formar parte de una copia de seguridad.

El FileSet del cliente Windows fue configurado para incluir los directorios de usuarios y programas instalados, excluyendo rutas temporales y la papelera de reciclaje:

```
# FileSet para el cliente Windows
FileSet {
  Name = "Windows Set"
  Include {
    Options {
      signature = MD5
      compression = GZIP
      IgnoreCase = yes
    }
    File = "C:/Users"
    File = "C:/Program Files"
  }
  Exclude {
    File = "C:/Windows/Temp"
    File = "C:/Temp"
    File = "C:/$Recycle.Bin"
  }
}
```

La opción `IgnoreCase = yes` fue habilitada para que Bacula trate las rutas de Windows sin distinción entre mayúsculas y minúsculas, evitando posibles problemas de coincidencia de rutas en el sistema de ficheros NTFS.

La estrategia semanal configurada en el Director fue la siguiente:

- Lunes: copia Full
- Martes: copia Incremental
- Miércoles: copia Differential
- Jueves: copia Incremental
- Viernes: copia Incremental
- Sábado: copia Incremental
- Domingo: copia Incremental

```
Schedule {
  Name = "WeeklyCycle"
  Run = Full mon at 23:05
  Run = Incremental tue at 23:05
  Run = Differential wed at 23:05
  Run = Incremental thu at 23:05
  Run = Incremental fri at 23:05
  Run = Incremental sat at 23:05
  Run = Incremental sun at 23:05
}
```

También fue necesario configurar correctamente el recurso de almacenamiento para que los clientes pudieran enviar los datos al servidor. Finalmente, el Storage quedó apuntando a la IP real del servidor y al dispositivo de almacenamiento correcto, evitando el uso de `localhost` y de dispositivos de ejemplo no válidos.

Configuración del File Daemon del servidor

En el servidor también se revisó el archivo `/etc/bacula/bacula-fd.conf`, correspondiente al File Daemon local. Este servicio permite que el propio servidor pueda actuar como cliente de Bacula en operaciones locales, además de participar en tareas de restauración y mantenimiento.

La configuración principal utilizada fue la siguiente:

```
  GNU nano 7.2                    bacula-fd.conf
#

#
# List Directors who are permitted to contact this File daemon
#
Director {
  Name = ubuntu-dir
  Password = "dfIYqIvv-6tirwCRMe2NTl16CKd7lFf0H"
}

#
# Restricted Director, used by tray-monitor to get the
#   status of the file daemon
#
Director {
  Name = ubuntu-mon
  Password = "28IkNOWbErFzXpCyaAyCiLjndQGApFVsB"
  Monitor = yes
}

#
# "Global" File daemon configuration specifications
#
FileDaemon {                          # this is me
  Name = ubuntu-fd
  FDport = 9102                  # where we listen for the director
  WorkingDirectory = /var/lib/bacula
  Pid Directory = /run/bacula
  Maximum Concurrent Jobs = 20
  Plugin Directory = /usr/lib/bacula
  FDAddress = 127.0.0.1
}

# Send all messages except skipped files back to Director
Messages {
  Name = Standard
  director = ubuntu-dir = all, !skipped, !restored, !verified, !saved
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify
```

En esta configuración se autorizó al Director `ubuntu-dir` a comunicarse con el File Daemon del servidor mediante una contraseña compartida. El daemon quedó escuchando en el puerto `9102`, con dirección `127.0.0.1`, lo que indica que solo acepta conexiones locales en el propio servidor.

Configuración del Storage Daemon del servidor

También se configuró el archivo `/etc/bacula/bacula-sd.conf`, encargado de definir el Storage Daemon, es decir, el servicio responsable de almacenar físicamente las copias de seguridad.

La configuración principal fue la siguiente:

```
  GNU nano 7.2                    bacula-sd.conf
#

Storage {
  Name = ubuntu-sd
  SDPort = 9103
  WorkingDirectory = "/var/lib/bacula"
  Pid Directory = "/run/bacula"
  Plugin Directory = "/usr/lib/bacula"
  Maximum Concurrent Jobs = 20
  SDAddress = 0.0.0.0
  TLS Enable = no
  TLS Require = no
}

#
# List Directors who are permitted to contact Storage daemon
#
Director {
  Name = ubuntu-dir
  Password = "USuZGyzPYhIzufhapGJ8w7Oa-orOjdUYA"
  TLS Enable = no
  TLS Require = no
}

#
# Restricted Director, used by tray-monitor to get the
#   status of the storage daemon
#
Director {
  Name = ubuntu-mon
  Password = "qyJdZ2irkZxK17_oVgbnu3gx_jXFiy2ah"
  Monitor = yes
}

#
# Note, for a list of additional Device templates please
#  see the directory <bacula-source>/examples/devices
# Or follow the following link:
#  http://www.bacula.org/git/cgit.cgi/bacula/tree/bacula/examples/devices?h>

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```

```
Device {
  Name = FileStorage
  Media Type = File1
  Archive Device = /bacula/backups
  LabelMedia = yes;
  Random Access = Yes;
  AutomaticMount = yes;
  RemovableMedia = no;
  AlwaysOpen = no;
  Maximum Concurrent Jobs = 5
}
```

```
#
Messages {
  Name = Standard
  director = ubuntu-dir = all
}
```

En esta práctica se simplificó la configuración por defecto del Storage Daemon, eliminando la necesidad de usar autochangers virtuales y dispositivos de ejemplo que apuntaban a rutas inexistentes. Se dejó únicamente un dispositivo de almacenamiento válido, llamado `FileStorage`, con tipo de medio `File1` y ruta física `/bacula/backups`.

De esta forma, tanto el cliente Debian como el cliente Windows utilizaron el mismo almacenamiento de disco para guardar sus copias de seguridad, sin necesidad de definir un `Device` distinto para cada cliente.

Configuración del cliente Debian

En el cliente Debian se instaló el paquete necesario mediante:

```
apt install bacula-client
```

Posteriormente se editó el archivo `/etc/bacula/bacula-fd.conf` para permitir la conexión del Director del servidor.

La configuración aplicada fue la siguiente:

```
  GNU nano 7.2                    bacula-fd.conf *
#

#
# List Directors who are permitted to contact this File daemon
#
Director {
  Name = ubuntu-dir
  Password = "dfIYqIvv-6tirwCRMe2NTl16CKd7lFf0H"
}

#
# Restricted Director, used by tray-monitor to get the
#   status of the file daemon
#
Director {
  Name = ubuntu-mon
  Password = "28IkNOWbErFzXpCyaAyCiLjndQGApFVsB"
  Monitor = yes
}

#
# "Global" File daemon configuration specifications
#
FileDaemon {                          # this is me
  Name = debian1
  FDport = 9102                  # where we listen for the director
  WorkingDirectory = /var/lib/bacula
  Pid Directory = /run/bacula
  Maximum Concurrent Jobs = 20
  Plugin Directory = /usr/lib/bacula
  FDAddress = 127.0.0.1
}

# Send all messages except skipped files back to Director
Messages {
  Name = Standard
  director = ubuntu-dir = all, !skipped, !restored, !verified, !saved
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify
```

Con esta configuración:

- El nombre del cliente quedó igual que en el servidor (`debian1`)
- La contraseña del bloque `Director` coincidió exactamente con la definida en el servidor para ese cliente
- El daemon quedó escuchando en el puerto `9102`
- Se permitió la escucha en todas las interfaces con `FDAddress = 0.0.0.0`

Comprobación de conectividad

Para verificar la conectividad entre servidor y cliente, se realizaron varias pruebas.

Desde el servidor se comprobó que el cliente Debian aceptaba conexiones en el puerto `9102`:

```
ubuntu@ubuntu:/etc/bacula$ nc -vz 192.168.1.143 9102
Connection to 192.168.1.143 9102 port [tcp/bacula-fd] succeeded!
ubuntu@ubuntu:/etc/bacula$
```

En el cliente Debian se verificó que el File Daemon estaba escuchando correctamente:

```
abel@debian1:~$ ss -tulpn | grep 9102
tcp   LISTEN 0      50
0.0:*
abel@debian1:~$
```

También se comprobó desde `bconsole` que el servidor podía contactar con el cliente:

```
* status client=debian1
```

La respuesta del daemon confirmó que la comunicación era correcta.

```
ubuntu@ubuntu:/etc/bacula$ sudo systemctl restart bacula-director
ubuntu@ubuntu:/etc/bacula$ sudo systemctl restart bacula-sd
ubuntu@ubuntu:/etc/bacula$ sudo bconsole
Connecting to Director 192.168.1.142:9101
1000 OK: 10002 ubuntu-dir Version: 13.0.4 (12 February 2024)
Enter a period to cancel a command.
*status
Status available for:
     1: Director
     2: Storage
     3: Client
     4: Scheduled
     5: Network
     6: All
Select daemon type for status (1-6): 3
The defined Client resources are:
     1: debian1
     2: windows-client-fd
Select Client (File daemon) resource (1-2): 1
Connecting to Client debian1 at 192.168.1.143:9102

debian1 Version: 15.0.3 (25 March 2025)  x86_64-pc-linux-gnu debian trixie/s
id
Daemon started 06-mar-26 21:22. Jobs: run=0 running=0 max=20.
 Heap: heap=712,704 smbytes=198,060 max_bytes=198,061 bufs=101 max_bufs=101
 Sizes: boffset_t=8 size_t=8 debug=0 trace=0 mode=0,0 bwlimit=0kB/s
 Crypto: fips=N/A crypto=OpenSSL 3.5.0 8 Apr 2025
 Plugin: bpipe(2)

Running Jobs:
Director connected using TLS at: 06-mar-26 21:22
No Jobs running.
====

Terminated Jobs:
====
You have messages.
*
```

Verificación de funcionamiento y ejecución de la copia de seguridad

Una vez corregida la configuración, se lanzó manualmente el job desde `bconsole`:

```
* run job=BackupClientLinux yes
```

```
* run job=BackupWindows yes
```

Posteriormente se comprobó el resultado con:

```
* list jobs
```

Resultado final:

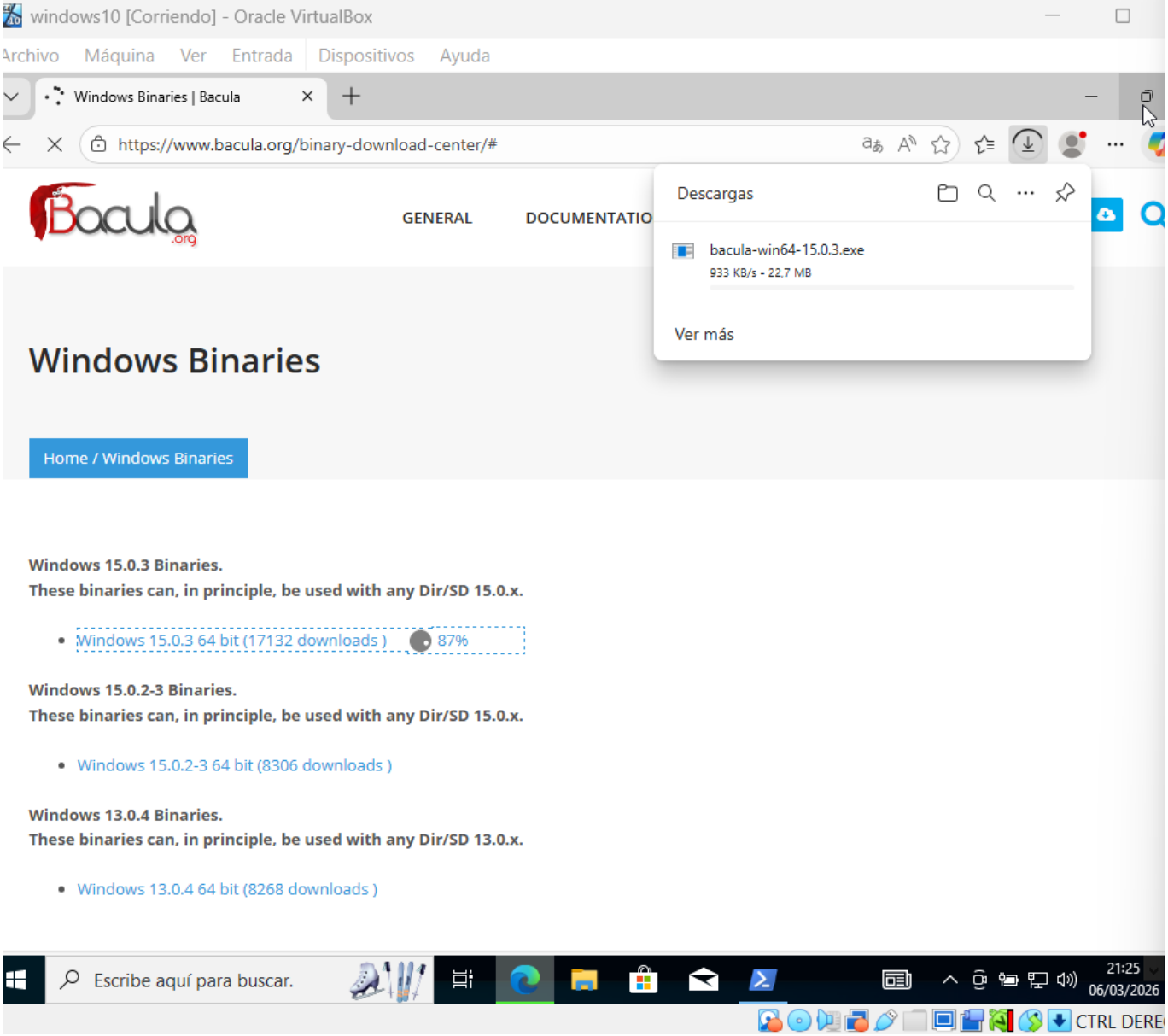
```
====

Terminated Jobs:
 JobId  Level     Files      Bytes   Status   Finished        Name
====================================================================
     1  Full       3,371    86.40 M  OK       06-mar-26 20:36 BackupClientLinux
     2  Full      15,312    838.1 M  OK       06-mar-26 20:42 BackupClientWindows

====
```

Con ello quedó verificado que:

- El servidor Bacula se comunica correctamente con el cliente Debian y Windows
- El cliente Debian y Windows se comunica correctamente con el Storage Daemon
- La copia Full del cliente Linux y el cliente Windows se ejecutó de forma satisfactoria

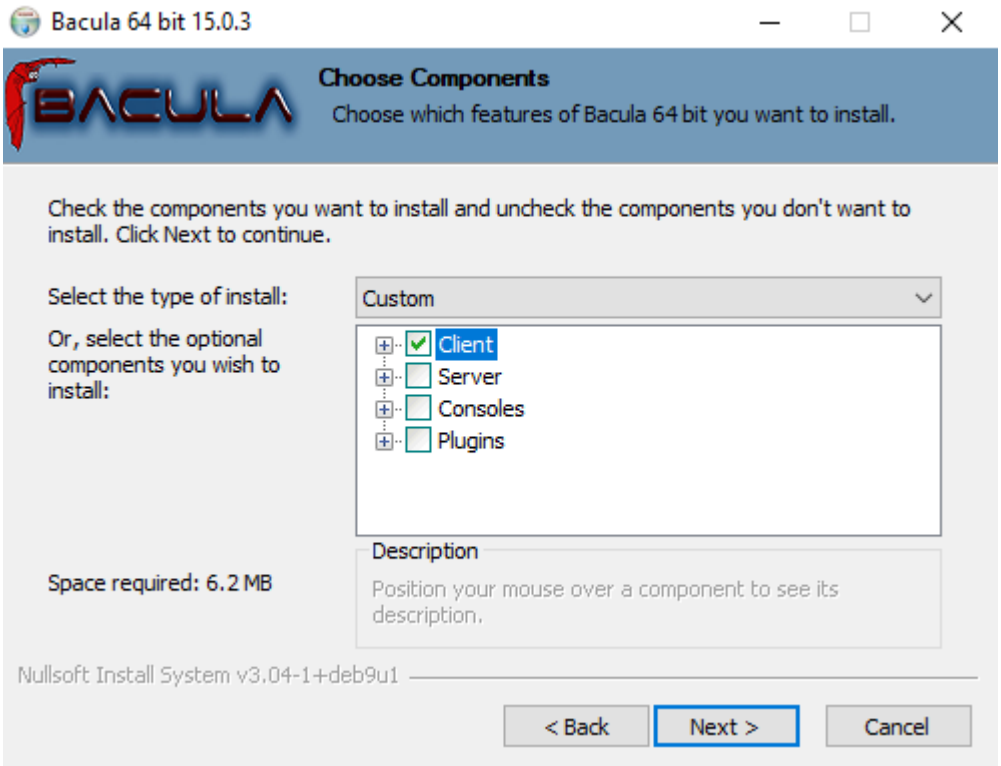
Instalación del cliente Bacula en Windows

Para incorporar un equipo Windows al sistema de copias de seguridad, se instaló el **cliente de Bacula (File Daemon)** en la máquina Windows. Este componente permite que el servidor Bacula se conecte al equipo cliente y realice la copia de los archivos configurados.



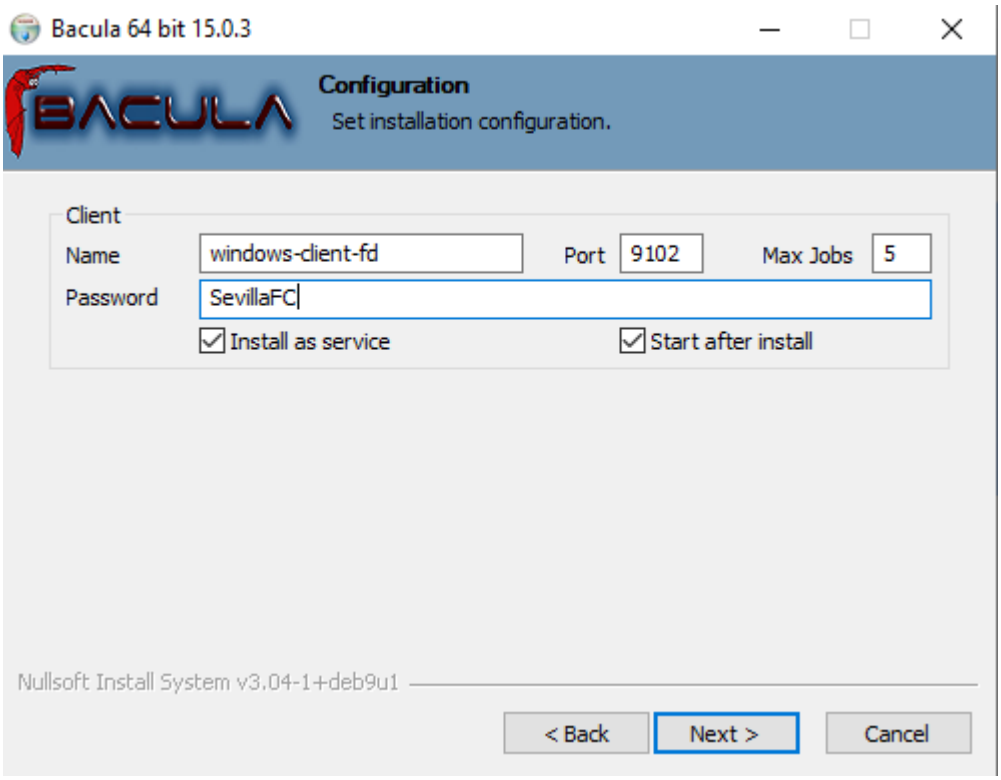
Instalación del software

En primer lugar, se ejecutó el instalador de Bacula para Windows. Durante el asistente de instalación se seleccionó la opción **Custom**, y dentro de ella se marcó únicamente el componente **Client**, ya que el objetivo era instalar solo el agente cliente en el equipo Windows.



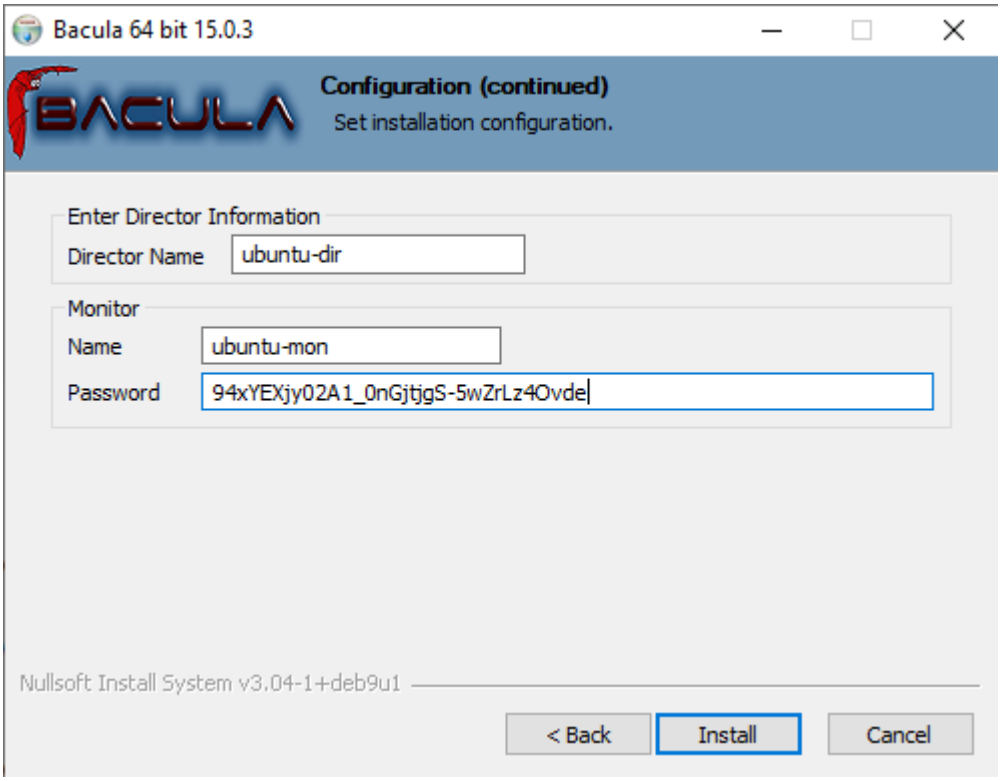
Durante la instalación se configuraron los siguientes parámetros principales:

- **Name**: nombre identificativo del cliente Windows.
- **Port**: puerto de escucha del File Daemon, manteniendo el valor por defecto `9102`.
- **Max Jobs**: número máximo de trabajos concurrentes permitidos.
- **Password**: contraseña compartida con el servidor Bacula.
- **Install as service**: activado, para instalar el cliente como servicio de Windows.
- **Start after install**: activado, para iniciar el servicio automáticamente tras la instalación.



Configuración adicional del instalador

En la última pantalla del instalador se solicitó la información del **Director**. En este apartado se introdujo el nombre del Director configurado en el servidor:



Además, el asistente generó automáticamente un nombre y una contraseña para el Monitor de Windows. Estos valores se pueden dejar por defecto, ya que el monitor no era imprescindible para la realización de la copia de seguridad, aunque yo le he modificado y puesto correctamente, debe de ser la misma contraseña que en `Console` dentro de `Bacula-dir.conf`.

Consideraciones importantes

Para que la instalación funcionase correctamente, fue necesario tener en cuenta los siguientes aspectos:

- El nombre del cliente en Windows debía coincidir exactamente con el configurado en el servidor.
- La contraseña introducida en el instalador debía ser la misma que la definida en `bacula-dir.conf`.
- El puerto `9102/TCP` debía estar accesible en el firewall de Windows.
- El servicio `bacula-fd` debía quedar iniciado tras la instalación.

Incidencia encontrada

Durante las primeras pruebas apareció un error relacionado con el dispositivo de almacenamiento:

```
Storage daemon "File1" didn't accept Device "FileChgr1" because: Device
"FileChgr1" not in SD Device resources or no matching Media Type
```

El problema no estaba en la instalación del cliente Windows, sino en una incoherencia entre la configuración del Director y la del Storage Daemon del servidor. En `bacula-dir.conf` se estaba utilizando el dispositivo `FileChgr1`, mientras que en `bacula-sd.conf` el dispositivo realmente definido era `FileStorage`.

Solución aplicada

La solución consistió en corregir el recurso `Storage` del servidor para que utilizase el dispositivo real configurado en el Storage Daemon:

```
Storage {
  Name = File1
  Address = 192.168.1.142
  SDPort = 9103
  Password = "**"
  Device = FileStorage
  Media Type = File1
  Maximum Concurrent Jobs = 10
  TLS Enable = no
  TLS Require = no
}
```

Una vez realizado este cambio y reiniciados los servicios de Bacula, el backup del cliente Windows pudo iniciarse correctamente.

Resultado

Con esta configuración, el cliente Bacula en Windows quedó correctamente instalado, registrado en el servidor y preparado para ejecutar copias de seguridad remotas desde el Director.

Restauración de un archivo en el cliente `debian1`

Para verificar el funcionamiento de la restauración en Bacula, se realizó una prueba práctica consistente en eliminar previamente un archivo del cliente Debian y recuperarlo posteriormente desde las copias de seguridad almacenadas en el servidor.

1. Eliminación previa del archivo

En el cliente `debian1` se eliminó manualmente el archivo:

```
/var/backups/dpkg.status.1.gz
```

El objetivo de esta prueba fue comprobar que Bacula era capaz de localizar el fichero en el catálogo y restaurarlo correctamente a partir de las copias realizadas anteriormente.

2. Acceso a bconsole

La restauración se inició desde el servidor Bacula accediendo a la consola de administración:

```
sudo bconsole
```

Una vez dentro, se ejecutó el comando:

```
restore
```

Este comando permitió lanzar el asistente interactivo de restauración de Bacula.

3. Selección del origen de la restauración

Dentro del menú de restauración se eligió la opción:

```
5: Select the most recent backup for a client
```

Posteriormente se seleccionó el cliente:

```
debian1
```

Bacula identificó automáticamente los JobId válidos más recientes asociados a este cliente y al FileSet `Linux Set`, seleccionando los siguientes trabajos:

- JobId 3 → copia Full
- JobId 6 → copia Incremental

De esta manera, Bacula construyó el árbol de directorios utilizando tanto la copia completa como la incremental, permitiendo recuperar el estado más reciente del sistema.

4. Selección del archivo a restaurar

Una vez cargado el árbol de archivos, se accedió al directorio donde se encontraba el fichero eliminado:

```
cd /var/backups
```

A continuación se listó el contenido del directorio para verificar que el archivo existía en el catálogo:

```
ls
```

Después se marcó el archivo concreto para restaurar:

```
mark dpkg.status.1.gz
```

Una vez seleccionado, se finalizó la fase de selección con:

```
done
```

Bacula indicó entonces que se había seleccionado 1 archivo para ser restaurado.

5. Confirmación del job de restauración

Antes de ejecutar la restauración, Bacula mostró el resumen del job:

- **JobName**: RestoreFiles
- **Where**: /tmp/bacula-restores
- **Backup Client**: debian1
- **Restore Client**: debian1
- **Storage**: File1
- **Replace**: Always

Finalmente, se confirmó la ejecución con:

```
yes
```

Tras ello, Bacula creó el trabajo de restauración con:

```
JobId = 3
```

```
1 file selected to be restored.

Using Catalog "MyCatalog"
Run Restore job
JobName:         RestoreFiles
Bootstrap:       /var/lib/bacula/ubuntu-dir.restore.1.bsr
Where:           /tmp/bacula-restores
Replace:         Always
FileSet:         Linux Set
Backup Client:   debian1
Restore Client:  debian1
Storage:         File1
When:            2026-03-06 20:47:28
Catalog:         MyCatalog
Priority:        10
Plugin Options:  *None*
OK to run? (yes/mod/no): yes
Job queued. JobId=3
*
```

```
*list jobs
+-------+---------------------+---------------------+------+-------+----------+-------------+-----------+
| JobId | Name                | StartTime           | Type | Level | JobFiles | JobBytes    | JobStatus |
+-------+---------------------+---------------------+------+-------+----------+-------------+-----------+
|     1 | BackupClientLinux   | 2026-03-06 20:35:45 | B    | F     |    3,371 |  86,407,327 | T         |
|     2 | BackupClientWindows | 2026-03-06 20:36:35 | B    | F     |   15,312 | 838,136,099 | T         |
|     3 | RestoreFiles        | 2026-03-06 20:47:34 | R    | F     |        1 |      91,875 | T         |
+-------+---------------------+---------------------+------+-------+----------+-------------+-----------+
*|
```

6. Ejecución de la restauración

Durante la ejecución del job, Bacula informó de los siguientes pasos:

- Restauración de archivos a partir de los JobId 3 y 6
- Conexión correcta con el Storage Daemon
- Uso del dispositivo `FileStorage` para lectura
- Conexión correcta con el cliente `debian1`
- Lectura del volumen `Vol-0001`

Todo ello confirmó que Bacula pudo acceder correctamente a la copia almacenada y reenviar el archivo al cliente Debian.

7. Resultado final

El resumen final del trabajo de restauración fue el siguiente:

- **Files Expected**: 1
- **Files Restored**: 1
- **Bytes Restored**: 91,875
- **FD Errors**: 0
- **SD termination status**: OK
- **Termination**: Restore OK

Esto confirmó que la restauración se completó correctamente y sin errores.

8. Ubicación del archivo restaurado

El archivo no fue restaurado directamente en su ruta original, sino en el directorio alternativo definido en el job de restauración:

```
/tmp/bacula-restores
```

Por tanto, la ruta esperada del archivo restaurado fue:

```
/tmp/bacula-restores/var/backups/dpkg.status.1.gz
```

Este comportamiento es recomendable, ya que permite verificar primero el contenido restaurado antes de copiarlo manualmente a su ubicación original.

9. Comprobación posterior

Una vez finalizado el restore, se pudo comprobar la existencia del archivo restaurado con:

```
root@debian1:/tmp/bacula-restores/var/backups# ls
dpkg.status.1.gz
root@debian1:/tmp/bacula-restores/var/backups#
```

Si se deseaba devolver el fichero a su ubicación original, podía hacerse manualmente con:

```
sudo cp /tmp/bacula-restores/etc/backups/dpkg.status.1.gz /etc/backups/
```

10. Ficheros Bacula Server:

Se adjuntaron los 3 ficheros `bacula-dir - bacula-sd y bacula-fd` al repositorio para mayor visibilidad.

Instalación de Baculum API y Baculum Web

Con el objetivo de disponer de una interfaz web para la administración de Bacula, se procedió a la instalación de *Baculum API* y *Baculum Web* sobre el servidor Ubuntu. Baculum permite gestionar trabajos de copia de seguridad, restauraciones, acceso al catálogo y, opcionalmente, la configuración de los componentes de Bacula desde entorno web.

Repositorio de Baculum

Como los paquetes de Baculum no se encontraban en los repositorios estándar del sistema, fue necesario añadir previamente el repositorio oficial de Baculum y su clave pública.

Para ello se ejecutó el siguiente comando para importar la clave del repositorio:

```
ubuntu@ubuntu:/etc/bacula$ sudo su
root@ubuntu:/etc/bacula# wget -qO - http://www.bacula.org/downloads/baculum/
baculum.pub | apt-key add -
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instea
d (see apt-key(8)).
OK
root@ubuntu:/etc/bacula#
```

A continuación, se añadió el repositorio correspondiente a Bacula Director 11 o superior:

```
  GNU nano 7.2                                        /etc/apt/sources.list.d/baculum.list
deb [ arch=amd64 ] http://www.bacula.org/downloads/baculum/stable-11/ubuntu focal main
deb-src http://www.bacula.org/downloads/baculum/stable-11/ubuntu focal main
```

Estas líneas se incorporaron al sistema de repositorios APT en el fichero: /etc/apt/sources.list.d/baculum.list

Una vez añadido el repositorio, se actualizó la información de paquetes disponible.

Instalación de Baculum API

El primer componente instalado fue Baculum API, que actúa como backend de comunicación entre Baculum Web y Bacula.

La instalación se realizó con:

```
sudo apt-get install baculum-common baculum-api baculum-api-apache2
```

Tras la instalación, fue necesario habilitar el módulo rewrite de Apache:

```
sudo a2enmod rewrite
```

También se habilitó el sitio virtual correspondiente a la API de Baculum:

```
sudo a2ensite baculum-api
```

Finalmente, se reinició Apache para aplicar la nueva configuración:

```
sudo systemctl restart apache2
```

Una vez completado este proceso, la API de Baculum quedó accesible en el puerto: http://IP_DEL_SERVIDOR:9096

Instalación de Baculum Web

Posteriormente se instaló la interfaz gráfica Baculum Web, que es el frontend desde el cual se administran los trabajos de Bacula.

La instalación se realizó mediante:

```
sudo apt-get install baculum-common baculum-web baculum-web-apache2
```

Igual que en el caso anterior, fue necesario habilitar el módulo rewrite de Apache:

```
sudo a2enmod rewrite
```

También se habilitó el sitio virtual de Baculum Web: sudo a2ensite baculum-web

Por último, se reinició Apache:

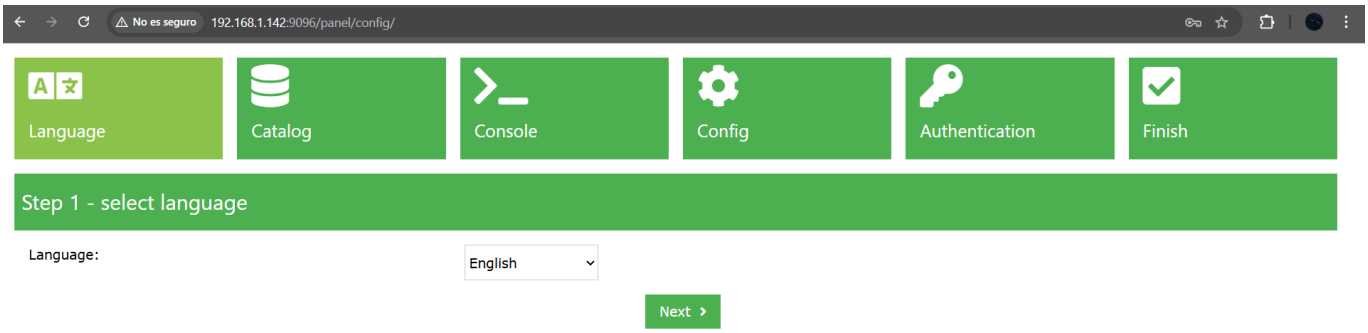
```
sudo systemctl restart apache2
```

Una vez finalizada la instalación, la interfaz web quedó accesible en: http://IP_DEL_SERVIDOR:9095

Configuración inicial de Baculum API

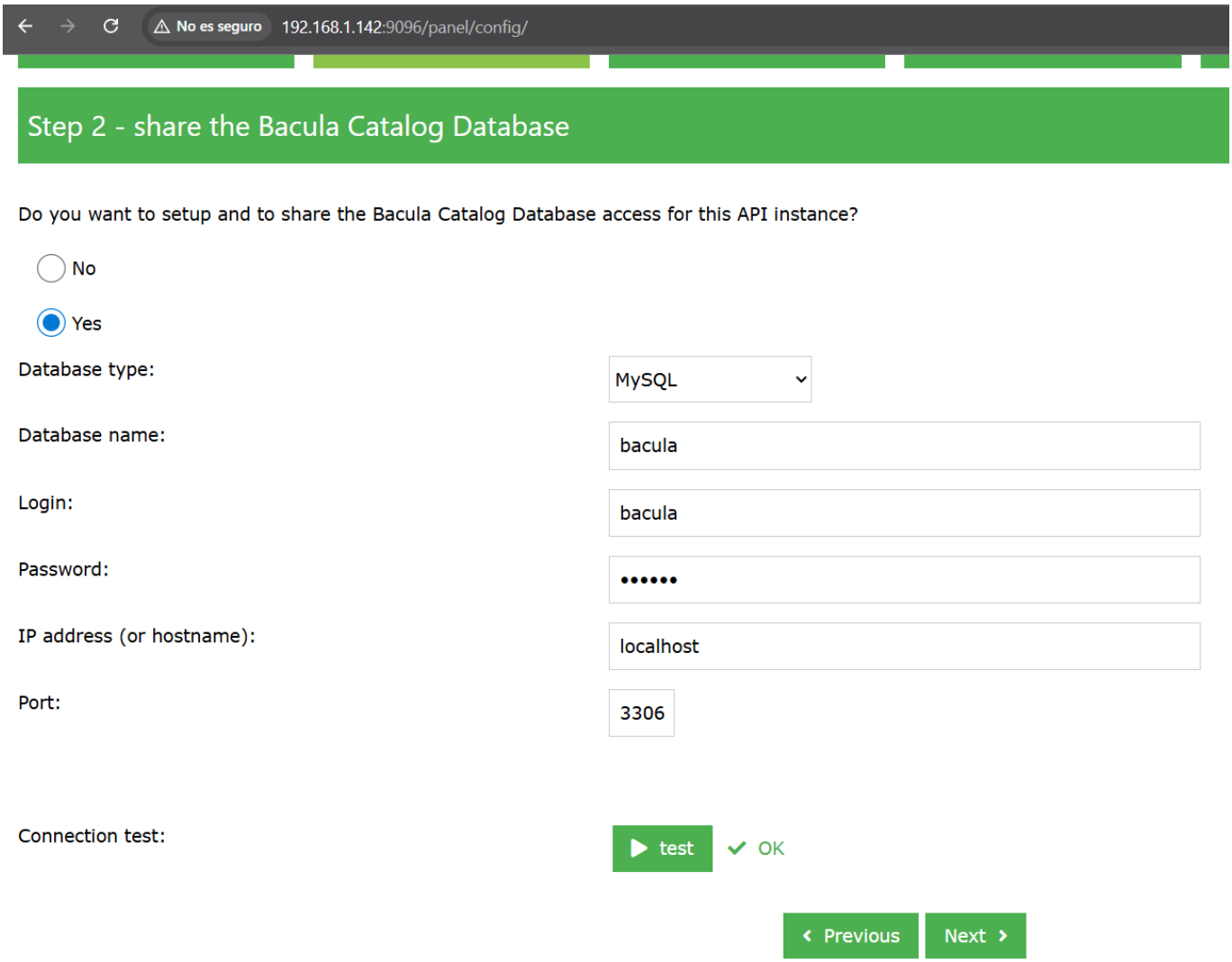
Antes de utilizar Baculum Web fue necesario configurar primero la API, ya que la interfaz web depende de ella para conectarse con Bacula.

La configuración se realizó accediendo mediante navegador a:



En el asistente de configuración inicial se definieron los siguientes elementos:

- Método de autenticación de la API.



- Acceso al catálogo de Bacula.
- Acceso a bconsole.
- Acceso opcional al módulo de configuración de Bacula.

Durante esta fase se seleccionó la autenticación HTTP Basic, que permite a Baculum Web comunicarse con la API mediante usuario y contraseña.

También se configuró el acceso a la base de datos del catálogo existente de Bacula, utilizando los mismos parámetros ya definidos en el bacula-dir.conf del servidor, de forma que Baculum reutilizase la base de datos ya existente en lugar de crear una nueva.

Configuración del acceso a bconsole

Para que Baculum API pudiera ejecutar comandos de Bacula a través de bconsole, fue necesario permitir al usuario del servidor web (www-data en Ubuntu con Apache) ejecutar dicho binario mediante sudo sin contraseña.

Para ello se creó un fichero de configuración en:

```
/etc/sudoers.d/baculum-api
```

- El contenido añadido fue el siguiente:

```
Defaults:www-data !requiretty
www-data ALL = (root) NOPASSWD: /usr/sbin/bconsole
www-data ALL = (root) NOPASSWD: /usr/sbin/bdirjson
www-data ALL = (root) NOPASSWD: /usr/sbin/bsdjson
www-data ALL = (root) NOPASSWD: /usr/sbin/bfdjson
www-data ALL = (root) NOPASSWD: /usr/sbin/bbconsjson
```

Este fichero se creó utilizando visudo para validar su sintaxis de forma segura:

```
sudo visudo -f /etc/sudoers.d/baculum-api
```

Posteriormente se corrigieron sus permisos para cumplir con los requisitos de sudoers:

```
sudo chown root:root /etc/sudoers.d/baculum-api
sudo chmod 0440 /etc/sudoers.d/baculum-api
sudo visudo -c
```

```
root@ubuntu:/etc/bacula# sudo chown root:root /etc/sudoers.d/baculum-api
sudo chmod 0440 /etc/sudoers.d/baculum-api
sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/baculum-api: parsed OK
root@ubuntu:/etc/bacula#
```

Con ello quedó autorizado el acceso a bconsole desde Baculum API.

Permisos para gestión de configuración

```
sudo usermod -aG bacula www-data
sudo chown bacula:bacula /etc/bacula/bacula-dir.conf /etc/bacula/bacula-sd.conf
/etc/bacula/bacula-fd.conf /etc/bacula/bconsole.conf
sudo chmod 660 /etc/bacula/bacula-dir.conf /etc/bacula/bacula-sd.conf
/etc/bacula/bacula-fd.conf /etc/bacula/bconsole.conf
sudo chown root:bacula /etc/bacula
sudo chmod 775 /etc/bacula
sudo systemctl restart apache2
```

Con estos cambios, Baculum pudo acceder correctamente a los ficheros de configuración del entorno Bacula.

Configuración del catálogo de Bacula

Durante el asistente de Baculum API también se configuró el acceso al catálogo de Bacula utilizando los datos ya existentes del servidor.

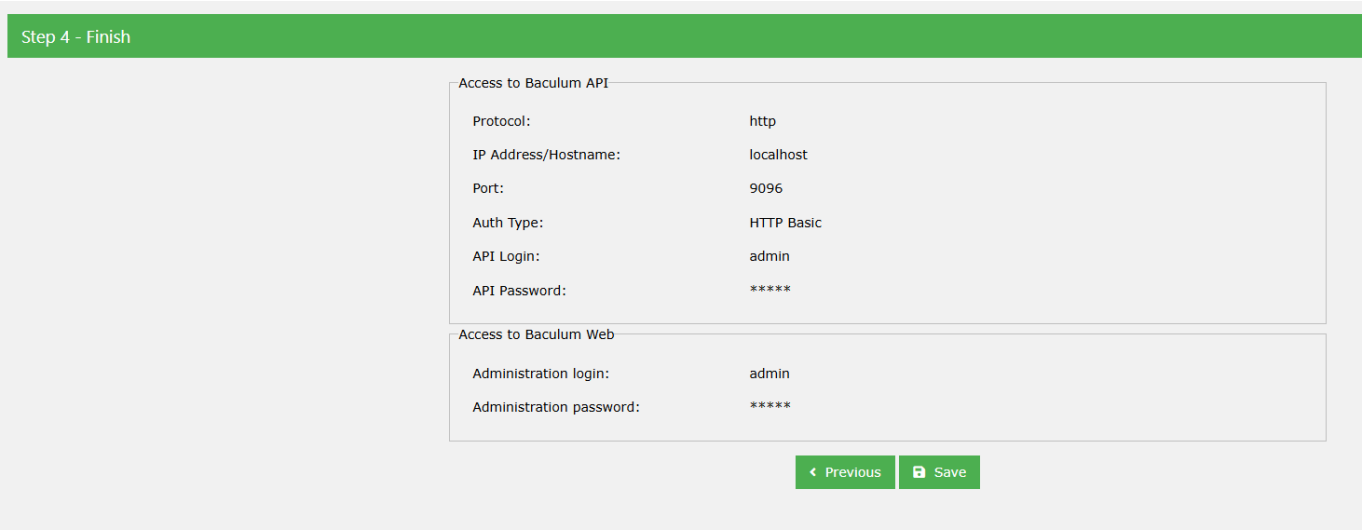
Los parámetros introducidos fueron los siguientes:

- *Database type*: MySQL
- *Database name*: bacula
- *Login*: bacula
- *Password*: contraseña del usuario de base de datos bacula
- *Host*: localhost
- *Port*: 3306

De esta manera, Baculum se conectó a la misma base de datos donde Bacula ya almacenaba el historial de copias de seguridad, volúmenes, trabajos y archivos respaldados.

Configuración de Baculum Web

Una vez configurada correctamente la API, se accedió a Baculum Web:



En el primer acceso se utilizó la autenticación de la API configurada previamente y, desde la sección de conexión con la API, se registró el host local con los siguientes parámetros:

- *Protocol*: HTTP
- *IP Address / Hostname*: 192.168.1.142 (IP DEL SERVIDOR)
- *Port*: 9096
- *Authentication*: HTTP Basic
- *API Login*: usuario configurado en la API
- *API Password*: contraseña configurada en la API

Una vez validada la conectividad, Baculum Web quedó enlazado con la API y pudo comenzar a mostrar la información del entorno Bacula.

Resultado final

Tras completar la instalación y configuración de ambos componentes, se obtuvo una interfaz web plenamente funcional para la administración de Bacula. ImagenFinal

Con Baculum Web fue posible:

- Consultar clientes, jobs, volúmenes y catálogo.
- Ejecutar copias de seguridad desde la interfaz web.
- Supervisar el estado de los trabajos.
- Gestionar restauraciones.
- Acceder, opcionalmente, a la configuración de Bacula desde entorno gráfico.

De este modo, el sistema Bacula quedó complementado con una interfaz web centralizada, facilitando la administración de las copias de seguridad sin depender exclusivamente de bconsole.